

TP1 — Git en local

Suivre l'historique d'un projet, valider des versions, corriger ses erreurs

Présentation

Un logiciel de contrôle de version (VCS : Version Control System) est un outil permettant de suivre l'évolution dans le temps d'un ensemble de fichiers, mais aussi de permettre à plusieurs personnes de travailler sur le même projet en fusionnant leurs modifications et, au besoin, en résolvant les conflits.

Ces logiciels peuvent suivre le cycle de vie de n'importe quel projet, mais ils sont surtout utilisés par les développeurs, quel que soit le type d'application (système, web, desktop...) et le langage (JavaScript, C#, Java, PHP...).

Il existe trois grandes familles de logiciels de contrôle de version :

Locaux

Adapté au suivi d'un projet par une seule personne. Permet de gérer les versions et de revenir en arrière si besoin.

Centralisés

Un serveur unique abrite le projet ; chaque développeur y stocke directement ses modifications. Le serveur gère la fusion et les conflits.

Distribués

Chaque développeur possède une copie complète du projet en local, valide son travail, puis le pousse (push) vers un serveur qui fusionne les contributions de tous. Pour rejoindre le projet, on tire (pull) une copie complète.

Git : un VCS distribué

Git est le VCS distribué le plus utilisé. Dans ce TP, vous travaillerez uniquement en local : il n'y a donc pas encore de push ni de pull, mais vous poserez déjà les bases (init, add, commit, log, diff, restore) qui serviront dès le TP2 pour collaborer via GitHub.

Contexte : travailler en ligne de commande

Vous allez développer un petit projet sous Git. Pour chaque étape, vous réaliserez les actions en ligne de commande, dans le terminal intégré de PhpStorm, et vous noterez les commandes saisies dans un fichier à déposer dans le répertoire indiqué par votre professeur.

Astuce — copier du texte depuis le terminal

- Sélectionnez le texte à copier.
- Appuyez sur la touche Entrée : cela copie la sélection (raccourci du terminal intégré de PhpStorm).
- Collez le texte dans votre document.

Sur un autre terminal (Git Bash, PowerShell...), le raccourci habituel est plutôt **Ctrl+Maj+C** pour copier.

Avant de commencer

Si ce n'est pas déjà fait, indiquez à Git qui vous êtes et alignez le nom de la branche par défaut sur celui utilisé par GitHub (**main** plutôt que **master**) :

```
git config --global user.name "Prénom Nom"
git config --global user.email "nom.prenom@lmdsio.fr"
git config --global init.defaultBranch main
```

Exercice 1 — Créer le dossier et initialiser le dépôt (git init)

1. Dans votre répertoire réseau (H:/), créez le dossier GitExo1.
2. Ouvrez PhpStorm puis ouvrez ce dossier GitExo1.
3. Ouvrez un terminal dans PhpStorm.
4. Placez-vous dans le dossier GitExo1 et exécutez :

```
git init
```

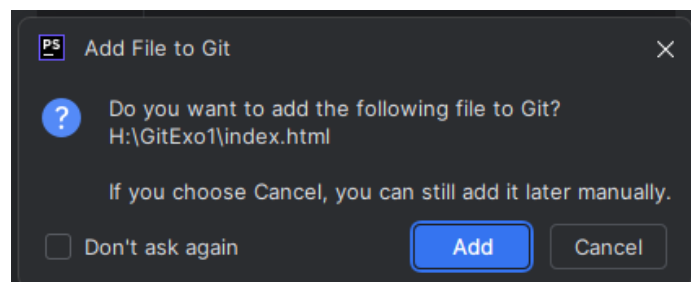
En affichant les éléments masqués de votre dossier GitExo1, vous pouvez voir qu'un dossier `.git` a été créé : c'est lui qui gère tout l'historique de votre code.

Aide en ligne : [Démarrer un dépôt Git](#)

Exercice 2 — Ajout de code source

Vous allez travailler sur un projet contenant du code HTML et PHP.

5. Dans PhpStorm, créez une page **index.html** dans votre dossier GitExo1. La fenêtre suivante devrait apparaître : cliquez sur **Add** pour ajouter le fichier aux fichiers suivis par Git.



6. Dans **index.html**, ajoutez le code affichant un titre de bienvenue et un lien vers la page **profil.html** que vous allez créer ensuite.

Balises nécessaires

- **<head>** contient les informations de la page, dont **<title>** (le titre affiché dans l'onglet du navigateur).
- **<body>** contient tout ce qui est visible sur la page.
- **<h1>...</h1>** affiche votre titre de bienvenue.
- **...** crée le lien vers `profil.html` ; le texte entre les balises est celui affiché et cliquable.

Squelette à compléter :

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>...</title>
</head>
<body>
  <h1>...</h1>
  <a href="profil.html">...</a>
</body>
</html>
```

7. Créez la page **profil.html** contenant un formulaire (**<form>**) pour saisir le prénom, le nom, et un bouton Valider qui envoie sur la page **accueil.php**.

Balises et attributs nécessaires

- **<form action="accueil.php" method="post">** ouvre le formulaire : action indique la page de destination, method="post" indique que les données voyageront sans apparaître dans l'URL (recommandé pour un formulaire).
- **<input type="text" name="prenom">** crée un champ de saisie. L'attribut name est essentiel : c'est lui qui permettra à accueil.php de retrouver la valeur saisie (à l'étape suivante). Faites de même pour le champ nom.
- **<input type="submit" value="Valider">** crée le bouton qui envoie le formulaire.

Squelette à compléter :

```
<form action="accueil.php" method="post">
  <label>Prénom : <input type="text" name="prenom"></label><br>
  <label>Nom : <input type="text" name="..."></label><br>
  <input type="submit" value="Valider">
</form>
```

8. Créez la page **accueil.php** qui récupère le nom et le prénom et affiche *Bonjour <prénom> <Nom>*.

Éléments de syntaxe PHP nécessaires

- **<?php ... ?>** délimite le code PHP au milieu du HTML.
- **\$_POST['prenom']** récupère la valeur saisie dans le champ du même nom. Comme le formulaire utilise method="post", on utilise \$_POST (avec method="get", ce serait \$_GET).
- **echo** affiche du texte. Une chaîne entre guillemets doubles peut intégrer directement une variable : "Bonjour \$prenom".

Squelette à compléter :

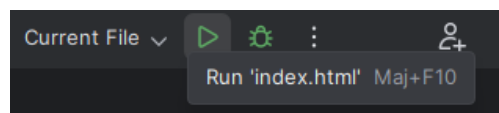
```
<?php
$prenom = $_POST['prenom'];
$nom = $_POST['...'];
echo "Bonjour ...";
?>
```

Pour aller plus loin

Un utilisateur pourrait saisir du code HTML dans le champ prénom. Pour s'en prémunir, on affiche en général une donnée saisie par l'utilisateur avec `htmlspecialchars($prenom)` plutôt que `$prenom` directement. Ce n'est pas demandé ici, mais c'est le réflexe à prendre dès que vous afficherez une saisie utilisateur.

Aide en ligne : [Syntaxe PHP](#) et [syntaxe HTML](#)

- Visualisez le résultat en cliquant sur la flèche verte (PhpStorm).



Exercice 3 — Premier ajout au suivi de version

- Ouvrez un terminal PhpStorm et exécutez `git status` pour afficher l'état des fichiers.

```
git status
```

Comprendre le message affiché

On **branch main** (ou **master** selon votre configuration) signifie que vous êtes sur cette branche — nous verrons ce qu'est une branche dans le TP2. **No commits yet** signifie que vous n'avez encore validé aucune version.

- Ajoutez toutes les pages à l'index de suivi avec `.` pour tout ajouter d'un coup :

```
git add .
```

- Validez cette version avec le message *"Mon projet version 1"* :

```
git commit -m "Mon projet version 1"
```

Convention pour la suite du TP

À partir de maintenant, chaque message de commit sera de la forme **"Mon projet version N"**, seul N changera.

- Affichez à nouveau le statut du projet :

```
git status
```

- Affichez l'historique des commits (l'option `--oneline` donne un résumé plus lisible) :

```
git log
git log --oneline
```

Aide en ligne : [Enregistrer des modifications dans le dépôt](#)

Exercice 4 — Modifications sur le code source

15. Modifiez votre code pour que le lien de `index.html` ne renvoie plus sur `profil.html` mais sur `login.html`, qui demande un login et un mot de passe et possède un bouton Valider renvoyant sur `accueil.php`.

Balises et attributs nécessaires

- Dans `index.html`, il suffit de changer la valeur de l'attribut `href` de votre lien : `<a href="login.html">...</a>`.
- `login.html` reprend la structure de `profil.html` : un `<form>` avec `action="accueil.php"` `method="post"`.
- `<input type="text" name="login">` pour le login.
- `<input type="password" name="mdp">` pour le mot de passe : `type="password"` est nouveau ici — il masque les caractères saisis à l'écran (affichés en ●●●), sans changer le fonctionnement du formulaire.
- `<input type="submit" value="Valider">` pour le bouton, comme dans `profil.html`.

Squelette à compléter :

```
<form action="..." method="...">
  <label>Login : <input type="text" name="..."></label><br>
  <label>Mot de passe : <input type="password" name="..."></label><br>
  <input type="submit" value="Valider">
</form>
```

16. Dans `accueil.php`, écrivez le code qui vérifie que le login est `"admin"` et le mot de passe `"azerty"`. Si c'est le cas, redirigez vers `profil.html`, sinon vers `index.html`.

Éléments de syntaxe nécessaires

- `if (condition) { ... } else { ... }` exécute le premier bloc si la condition est vraie, sinon le second.
- `==` compare deux valeurs (à ne pas confondre avec `=`, qui affecte une valeur à une variable).
- `&&` est le ET logique : il permet de vérifier que le login et le mot de passe sont corrects en une seule condition.
- `$_POST['login']` et `$_POST['mdp']` récupèrent les champs saisis dans `login.html` — les noms doivent correspondre exactement aux attributs `name` utilisés dans le formulaire.
- `header("Location: ...")` redirige vers une autre page (voir rappel ci-dessous) : placez cet appel dans chacune des deux branches du `if/else`, avec l'URL correspondante.

Squelette à compléter :

```
<?php
$login = $_POST['...'];
$mdp = $_POST['...'];

$host = $_SERVER['HTTP_HOST']; // on récupère le nom de l'hôte
$uri = rtrim(dirname($_SERVER['PHP_SELF']), '/\\'); // on récupère le début de
l'URL

if (...) {
    header("Location: http://$host$uri/profil.html"); // vers profil.html
} else {
    header("Location: http://$host$uri/..."); // vers index.html
}
?>
```

Attention à l'ordre

header() doit être appelé avant tout affichage HTML (même un simple espace ou retour à la ligne avant `<?php`) : sinon PHP renverra une erreur "headers already sent".

17. Dans le terminal, affichez les modifications réalisées sur chaque page :

```
git diff
```

Défilez le terminal pour lire les modifications, puis tapez :q pour quitter la visionneuse.

18. Ajoutez les fichiers puis validez la version 2 :

```
git add .
git commit -m "Mon projet version 2"
```

19. Affichez la liste des commits :

```
git log --oneline
```

Exercice 5 — Un bug (in)volontaire

20. Modifiez **accueil.php** en y introduisant une erreur de logique : si le login est *"admin"* et le mot de passe *"azerty"*, redirigez vers **index.html** (au lieu de **profil.html**), et sinon vers **profil.html** (au lieu d'**index.html**).

21. Ajoutez puis validez la version 3 :

```
git add .
git commit -m "Mon projet version 3"
```

22. Exécutez **git log** et copiez le hash du commit de la version 2 (une suite hexadécimale, par exemple 08fa2f9).

```
git log --oneline
```

23. Comparez la version 2 et la version 3 avec `git diff` en indiquant les deux hash (celui de la version 2, puis celui de la version 3) :

```
git diff hash_version_2 hash_version_3
```

Pourquoi préciser les deux hash ?

`git diff hash_version_2` (un seul hash) compare ce commit à l'état actuel de votre dossier de travail — cela ne fonctionne comme attendu que si HEAD est bien sur la version 3 et qu'il n'y a aucune modification non validée. Préciser les deux hash (`git diff A B`) compare exactement ces deux commits entre eux, quel que soit l'endroit où vous vous trouvez : c'est plus fiable.

Aide en ligne : [Visualiser l'historique des validations](#)

Exercice 6 — Des changements locaux non commités

24. Ajoutez un champ date de naissance dans la page `profil.html`.
25. Ajoutez cette modification à l'index git :

```
git add profil.html
```

26. Retirez la modification de l'index avec `git restore --staged` (remplacez `nom_fichier` par `profil.html`) :

```
git restore --staged nom_fichier
```

La modification n'est désormais plus dans l'index, mais le fichier contient toujours le champ ajouté.

27. Annulez complètement la modification et revenez à la dernière version validée avec `git restore` :

```
git restore nom_fichier
```

git restore, la commande moderne

Avant Git 2.23, on utilisait `git checkout -- nom_fichier` pour annuler les modifications d'un fichier. Cette syntaxe fonctionne toujours, mais elle prête à confusion car `git checkout` sert aussi à changer de branche. `git restore` est dédiée à la restauration de fichiers : plus explicite, elle est recommandée depuis plusieurs années.

Vous pouvez maintenant constater que le champ date de naissance a disparu de `profil.html`.

28. Affichez le statut du suivi du projet :

```
git status
```

Aide en ligne : [Annuler des actions](#)

Exercice 7 — Un autre bug et retour à une version qui fonctionne

29. Modifiez `accueil.php` en y introduisant une nouvelle erreur : en cas de succès, elle renvoie vers `login.html` au lieu de `index.html`.
30. Ajoutez puis validez la version 4, puis affichez la liste des versions :

```
git add .
git commit -m "Mon projet version 4"
git log --oneline
```

Les versions 3 et 4 comportent des erreurs. Vous souhaitez revenir à la version 2, sans le faire à la main : Git permet de revenir à une ancienne version du code.

31. Récupérez le hash de la version 2 avec **git log**, puis revenez à ce commit :

```
git reset --hard hash_de_la_version_2
```

⚠ Une commande à utiliser avec beaucoup de précaution

git reset --hard annule les commits suivants **et supprime définitivement** toutes les modifications de code associées : impossible de les récupérer ensuite (sauf technique avancée de type reflog). Ne l'utilisez jamais sans être certain de vouloir perdre ce travail.

Et sur un historique déjà partagé ?

Ici, tout reste local : réécrire l'historique avec `reset --hard` ne dérange personne d'autre. Dans le TP2, une fois vos commits envoyés (push) sur GitHub et potentiellement récupérés par d'autres, on préfère **git revert**, qui annule un commit en en créant un nouveau, sans réécrire l'historique existant.

Aide en ligne : [Revenir à une ancienne version](#)

Exercice 8 — Ignorer certains fichiers avec .gitignore

PhpStorm crée automatiquement un dossier `.idea/` contenant vos préférences personnelles d'éditeur (thème, mise en page des fenêtres...). Ces réglages n'ont aucun intérêt pour les autres développeurs et n'ont pas leur place dans l'historique du projet.

32. Vérifiez avec **git status** si le dossier `.idea/` apparaît comme fichier non suivi.

33. À la racine de GitExo1, créez un fichier nommé exactement `.gitignore` et ajoutez-y :

```
.idea/
*.log
```

34. Relancez **git status** : `.idea/` ne doit plus apparaître dans les fichiers non suivis.

35. Ajoutez et validez le fichier `.gitignore` lui-même :

```
git add .gitignore
git commit -m "Ajout du fichier .gitignore"
```

Si `.idea/` a déjà été commité par erreur

`.gitignore` n'agit que sur les fichiers pas encore suivis. Si `.idea/` a déjà été ajouté à un commit précédent, il faut d'abord l'en retirer explicitement, sans supprimer les fichiers de votre disque :

```
git rm -r --cached .idea
git commit -m "Retrait de .idea du suivi"
```

Pour aller plus loin

- [Learn Git Branching](#) — un simulateur visuel et interactif pour manipuler commits et branches.
- `git log --oneline --graph --all` — une vue compacte et graphique de l'historique, utile dès que plusieurs branches entrent en jeu (TP2).
- [Documentation officielle Git \(Pro Git, en français\)](#)